

Member ID: _____

Time: _____

Rank: _____



C# PROGRAMMING

(330)

NATIONAL 2023

PRODUCTION (90 minutes)

NationalBPA

_____ (600 points)

INDUSTRY CERTIFICATION (90 minutes)

IT Specialist Software Development

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. No equipment, supplies, or materials other than those specified for this event are allowed in the testing environment.
2. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing environment.
3. Your name and/or school name should NOT appear on any work you submit for grading. Use contestant # where appropriate.
4. You must submit your entire solution/project so that the graders may open your project to review the source code and/or build and execute your solution/project. **Submissions that do not contain source code will not be graded.**

PART ONE Instructions: Production

Once you have completed listed below, upload all competition materials (including project files, etc.) to the National BPA Pre-submit System at presubmit.bpa.org.

Compress all files in a single archive (zip file) and use the following naming convention for your documents:

CS-MemberID.zip

Development Standards

- Standard name prefixes must be utilized for variables.
- All subroutines, functions, and methods must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any).

PART TWO Instructions: Industry Certification

This exam requires IT Specialist Software Development Certification. Please do not leave your seat or exit the room until you have completed the certification. After the certification has been administered, you are permitted to leave.

Assumptions to make when taking this assessment:

- The form design is already created and does not need to be adjusted.
- There is input required by the user.
- The program will only exit when the VS stop button is manually clicked.
- All objects (buttons, list boxes, labels, panels, etc.) needed for the design aspect of the form have been provided.
- The needed graphic files for the pictures boxes are in the folder and are already configured.
- All required packages needed to complete have been provided for you.

Development Standards:

- Your Code must use a consistent variable naming convention.
- All subroutines (if any), functions (if any), and methods (if any) must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any). Readability is a goal of good code.

NationalBPA Overview

FIRST HALF

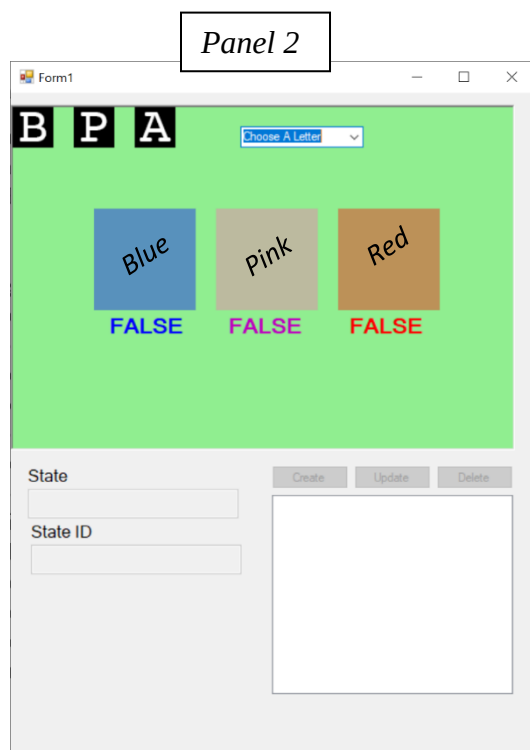
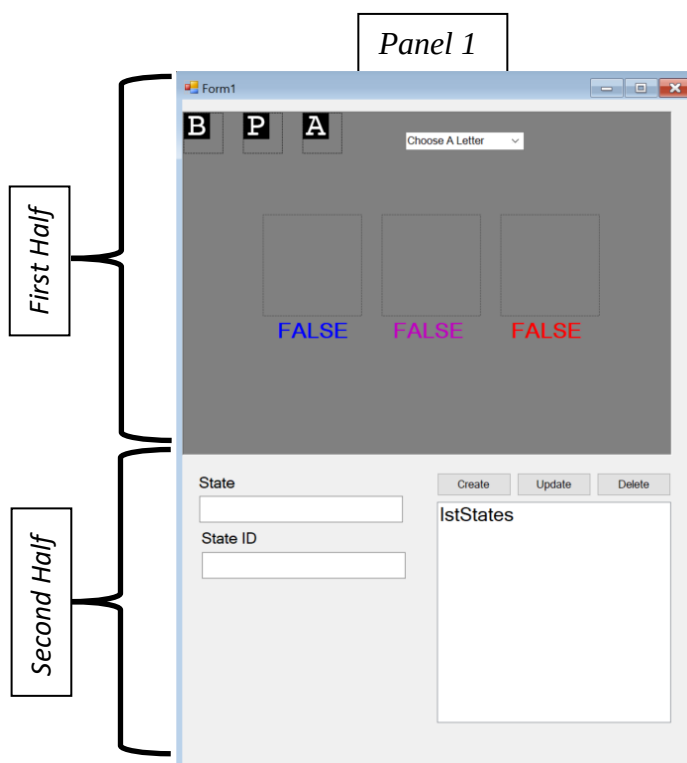
In this challenge, you will be required to create a program that will be broken into two parts. The first part will require that you create a physical passcode system similar to some of the CAPTCHA tests used to eliminate low level AI bots from gaining access to certain online software systems. **WARNING:** all the objects in the second half must be disabled in order for the controls in the first half to work. The user will be required to move three separate letters (B,P,A) to three distinct locations. Once the letter-keys has been moved to the precise location, the software will unlock the second half of the program. The letter-keys will be able to move with the arrow keys, no other input options are allowed (Up, Down, Right, Left). An interface has been provided that is required to be used for the creation of the key inputs for letter-key movements. In addition, there will be combo box that will allow the user to select which letter they would like to be able to move. The order of which letter-key is selected first is not important; what is of importance is that the letter-key moves and the program detects when it has arrived at its target location, which is another picture box. The user will be alerted that the letter-key has arrived by an adjustment of the color of the target location. The color adjustment will be an altering of the target-location alpha channel. The original alpha channel will be set to 100, and once the letter-key lands in the target-location the alpha channel will be set to its max opaque value of 255 (NOTE: alpha channels go from 0 to 255 and adjust a colors level of transparency). All of the picture box objects will be required to take place in a panel (provided). The letter-keys cannot move past the edge of the panel and must be programmed to stop when their edge reaches the panel with ZERO cross over. In addition, the letter-keys when accessing the target-location must completely be inside of its borders.

SECOND HALF

The second half of the program will require the users to utilize a dictionary storage system to store the states of BPA chapters. This half of the program will only be enabled once the first half has unlocked the passcode. The information for the states will be present in your starter files as simple hard coded text that you will process and store in a dictionary. No files will need to be read; however, the software will require that the end user can enter in their own states into this dictionary and have the program check for duplicates (duplicates are not allowed), and also give the user to physically select a state from the list and have it deleted. Dictionaries store two pieces of information: the key and a value. During this program, the key will be the state name, and the value will be the state abbreviation. For example, Montana (key) and MT (value). Do not concern yourself with why the state name is the key, and the abbreviation is the value. Message boxes will be used to inform the user when they successfully have completed an action in this second half.

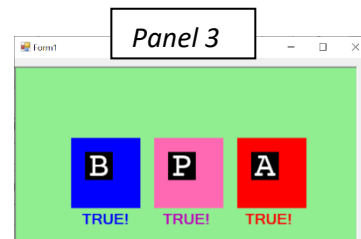
Input/Output

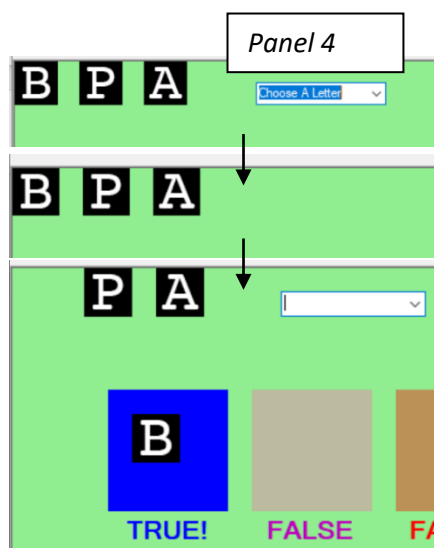
Panel 1 (First Half) displays the first half and second half in the design form. The BPA picture boxes are already designed to be a size of 40x40 pixels and will fill the space when the program runs. It is highly advisable to not alter the locations, sizes of the objects in the provided form. It should be noted that color of the panel is set to a dark gray so it can easily be identified. When the program runs, it should turn to another color. You will be required to use light green for its running color. The labels will change from FALSE to TRUE when the letter-key is in the proper location. The Second Half will be the location where the user creates the dictionary of states (click [“Create”] button) and will display in the list box.



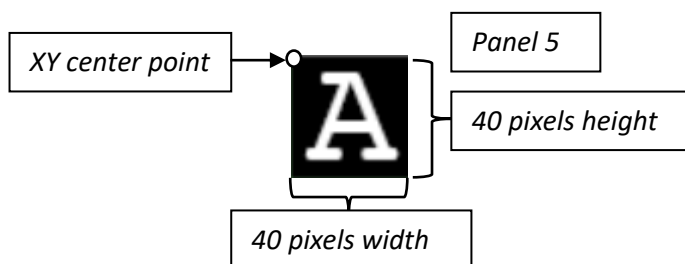
Panel 2 shows the program when it is running. NOTE: due to the probability of receiving a black and white copy of these instructions, the name of the color has been placed on the target-location for you. The alpha channels for these target-locations should be set to 100 in the code you will create. Also notice that all of the object in the second half have been disabled. The dropdown combo box will allow the user to pick which letter they want to move. If they choose ‘B’ then the letter-key that moves until completion will be the ‘B’.

Panel 3 shows the desired end goal of moving the three letters. B targets the blue picture box; P targets the pink picture box; A targets the red picture box. Notice how the labels change from FALSE to TRUE; and notice how the letters are clearly inside of the target and not on the edge.

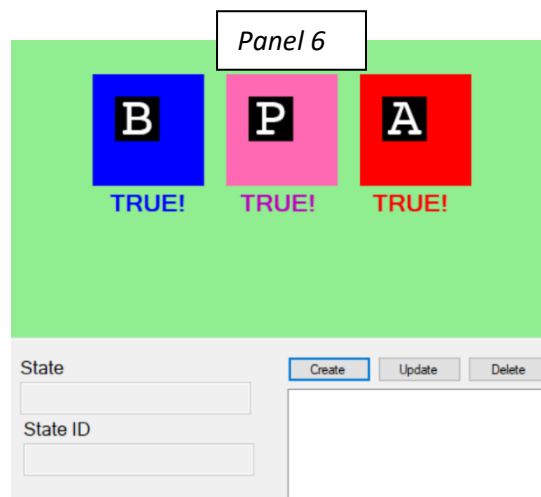




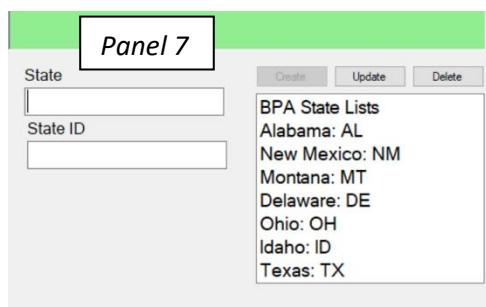
Panel 4 shows the combo box being turned to disabled it is not visible once a letter is selected. In this example ‘B’ is chosen and moved to its target location. The combo box will need to return to being visible and enabled once the letter reaches its target-location. This will need to happen for all letters being selected. Remember, the user can pick the letter-key. When the letters are moving, they will need to move in increments of 20 pixels along the X and Y axis in any direction (up, down, left, and right). The size of the letter-keys are 40x40 pixels and the location of the object is determined by its XY coordinate (**Panel 5**) which is the top left of the shape. The letters cannot move beyond the boundaries of the panel. The formula for movement will need to compensate that the width extends to right and the height extends downward.



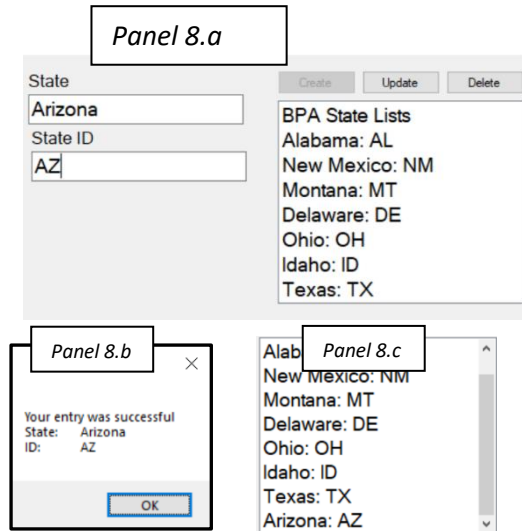
Panel 6 brings your attention to the [“Create”] button (second half) and how it is enabled after the targets each have the correct letter-key placed on them. Also, the combo box with letters on the drop-down list from the first half should still be disabled and not visible to avoid any confusion.



Panel 7 demonstrates how the dictionary list is created and prints to the list box. Also notice that the text entry boxes, and other buttons are enabled after the list is created. A text header is added to the list “BPA State Lists”.

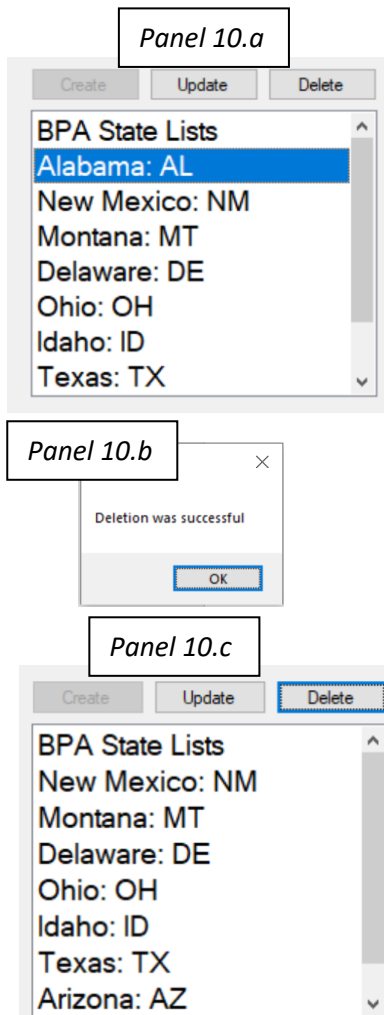


The [“Create”] button will be used one time. After it creates the dictionary list it will be disabled and never reactivated during the use of the form. In **Panel 7** you can see that is disabled.



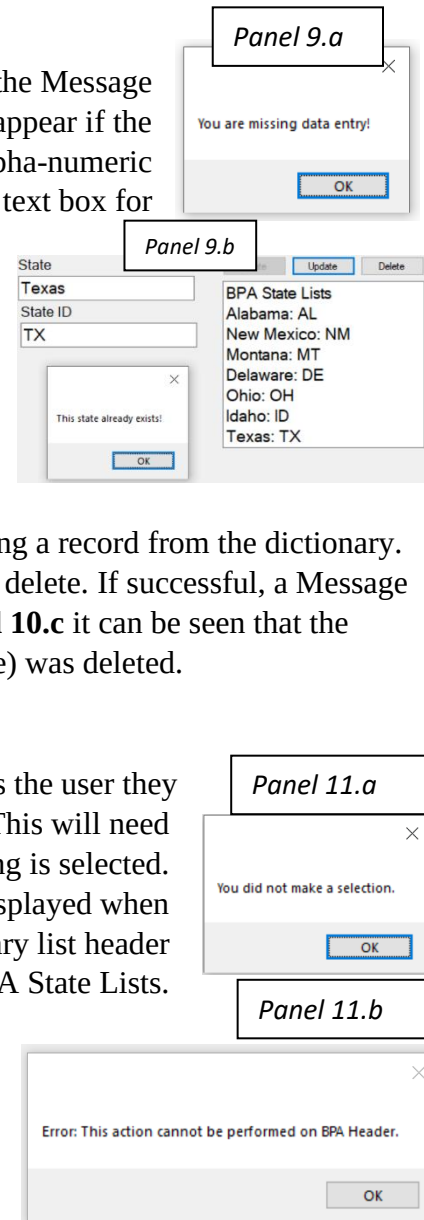
Panel 8.a demonstrates the Update button. The user enters in their state and abbreviation into the proper fields. Then the ["Update"] button is clicked, and it causes a Message Box to appear (**Panel 8.b**) letting the user know they successfully added the record. Then after the Message Box is closed the new entry is displayed at the bottom of the list box (**Panel 8.c**). NOTE: do not worry about spelling errors or making the dictionary in alphabetic order.

creating a new state. It is the same message for both text boxes. **Panel 9.b** shows another Message Box needs to be created to warn the user if they attempt to create a duplicate state. Texas is used in this example.



Panel 10.a shows the process for deleting a record from the dictionary. The user selects the entry, and then hits delete. If successful, a Message Box appears (**Panel 10.b**), and in **Panel 10.c** it can be seen that the selected entry (Alabama in this example) was deleted.

Panel 11.a is a Message Box that warns the user they did not make a selection for deletion. This will need to be programmed to detect if nothing is selected. **Panel 11.b** is another error that is displayed when the user attempts to delete the dictionary list header "BPA State Lists."



Requirements

1. You must use the provided project called “NationalBPA”. This project will contain a “Form1.cs[design]” that has been already created in the organization and naming conventions.
2. The file “Form1.cs” has already been provided. This file contains all needed packages and the method headers have already been provided. You will program within this file.
3. No additional buttons can be added to the form design.
4. Your contestant number must appear as a comment at the top of the main source code file.
5. User input is needed for this program as the program will gain functionality through user moving the letter-keys to the correct target-locations. The user will also interact with the buttons, combo box, and type in text.
6. You will be required to implement the given interface (controller.cs). The interface methods all have similar method signatures: **void MoveUp(PictureBox pb, bool cm);**
 - a. The picture box (**pb**) that is selected from the combo box will be used to determine which one to move.
 - b. The boolean argument is used to make sure they can move; there are multiple ways to use this variable so be creative.
7. If your program uses colors that make the text too difficult to read, those sections will not be scored. Be careful to use the required colors with alpha settings which have been predetermined to offer sufficient contrast.
8. The letter-keys need to move in increments of 20 pixels in the direction the arrows being pressed. Any movement greater than 20 pixels will result in loss of points. Remember that the Y increases going down.
9. All objects on the form have been named in advance and should not be change.
10. First Half Objects/Methods (provided)
 - a. Combo Box: **cmbLetters** Method Name: **cmbLetters_SelectedIndexChanged**
 - i. Stored Strings: “Choose A Letter”, “Letter A”, “Letter B”, “Letter P”
 - b. Target Picture Box Names: **pb_Target_A, pb_Target_B, pb_Target_P**
 - i. These will have their color alpha channels adjust based upon the key reaching them. Start at 100, change to 255.
 - c. Letter Picture Box Names: **pb_letter_A, pb_letter_B, pb_letter_P**
 - i. These will be programmed to moved by the user.
 - d. Labels Names: **lbl_B_Key, lbl_P_Key, lbl_A_Key**
 - i. The labels will be programmed to change their text from FALSE to TRUE when a letter-key reaches the appropriate target.
 - e. Panel Name: **panel1** this will be the boundaries of where the picture boxes can move
11. Second Half Objects/Methods (provided)
 - a. Button Name: **btnCreate** Method Name: **btnCreate_Click** Text: “Create”
 - i. This will add the hard coded texts to the dictionary
 - ii. It will also populate the list box with the items added to the dictionary
 - b. Buttons Name: **btnUpdate** Method Name: **btnUpdate_Click** Text: “Update”
 - i. This will add the newly typed records into the dictionary
 - ii. Checks for blank and duplicate entries and displays Message Box
 - c. Buttons Name: **btnDelete** Method Name: **btnDelete_Click** Text: “Delete”
 - i. Removes the selected items from the dictionary and the list box
 - ii. Checks for selecting nothing or selecting the BPA header
 - d. Method **HardCodeInfo** method is used to populate the dictionary

Solution and Project (There is NO partial credit)		
The project is present on the flash drive		10 points
Program Execution		
The program runs from the USB flash drive: the form loads with no initial errors		10 points
<i>If the program does not execute, then the remaining items in this section receive a score of zero.</i>		
Start Up: the combo box is active and can be accessed		10 points
Start Up: all labels below the target picture boxes are set to false		10 points
Start Up: the picture boxes with the letters BPA are present in that exact order.		10 points
Start Up: all objects from the second half of the program are not active		20 points
First time use: when any of the letters are selected from the combo box it is not visible;		10 points
When 'B' is selected it moves in four directions equally (all or nothing)		10 points
When 'P' is selected it moves in four directions equally (all or nothing)		10 points
When 'A' is selected it moves in four directions equally (all or nothing)		10 points
None of the letters can move off the panel where the letter breaches the edge of the panel (all or nothing)		30 points
Engaging Target: when the letters are clearly inside the target it changes colors (all or nothing)		30 points
Engaging Target: when the letters are clearly inside the target the label changes from FALSE to TRUE (all or nothing)		30 points
Second Half: the "Create" button of program unlocks when the letters are arranged in BPA, in that exact order on the targets.		20 points
Second Half: when ["Create"] button is pressed the list is displayed in the list box in the exact order.		20 points
Second Half: capable to add a state and abbreviation to the list with the ["Update"] button; Message Box must appear.		20 points
Second Half: Message Box displays if trying to enter a duplicate state (program must keep running).		20 points
Second Half: Message Box displays if trying to enter a blank state entry (program must keep running).		20 points
Second Half: can successfully delete an item from the list; Message Box must appear.		20 points
Second Half: Message Box displays if trying to delete with no selection (program must keep running).		20 points
Second Half: Message Box displays if trying to delete BPA header (program must keep running).		20 points
Subtotal		/360 Points

Source Code Review (There is NO partial credit)		
A comment containing the contestant number is present		10 points
Form1.cs class: implements and uses the “controller.cs” interface (four methods total). Code must demonstrate how movement is controlled to prevent movement beyond the boundaries. There also must be code that demonstrates how code is able to get the user key inputs; HINT: you will need to use a separate method for this. Comments are required; mark these comments as “SC1” at the beginning of the four implemented interface methods, plus a comment for the method that that gets the key inputs.		50 points
Form1.cs class: code showing how you set the alpha channels to 100 for the three pictures boxes that are the target-location. All or nothing. Comment is required; mark this comment as “SC2” at the beginning.		10 points
Form1.cs class: <i>cmbLetters_SelectedIndexChanged</i> method is commented explaining how the selection is made and how it is removed. The code must have worked to get credit. Mark this comment as “SC3” at the beginning.		10 points
Form1.cs class: <i>ComboOnOff</i> method code with comments showing how the combobox was enabled/visible and disabled/nonvisible in between selections. The code must have worked to get credit. Mark this comment as “SC4” at the beginning.		30 points
Form1.cs class: <i>Student created method</i> that is used to check if the letter-key is inside of the target-location. The code must have worked to get credit. Mark this comment as “SC5” at the beginning.		30 points
Form1.cs class: <i>Student created method</i> that activates the second half of the program once all three letter-keys have been marked as true in their target-location. The code must have worked to get credit. Mark this comment as “SC6” at the beginning.		10 points
Form1.cs class: <i>HardCodeInfo</i> method demonstrates how hard coded text is entered into a dictionary. The code must have worked to get credit. Mark this comment as “SC7” at the beginning.		10 points
Form1.cs class: <i>btnCreate_Click</i> method is commented explaining how it causes the dictionary records to display on the list box with the proper header “BPA State Lists”. The code must have worked to get credit. Mark this comment as “SC8” at the beginning.		20 points
Form1.cs class: <i>btnUpdate_Click</i> method is commented explaining how the code checks dictionary for duplicate and blank entries. You must use Dictionary method <i>Contains</i> . The code must have worked to get credit. Mark this comment as “SC9” at the beginning.		20 points
Form1.cs class: <i>btnDelete_Click</i> method is commented explaining how the code deals with <i>NullReferenceException</i> and <i>ArgumentOutOfRangeException</i> . Code must use at least 1 try/catch. The code must have worked to get credit. Mark this comment as “SC10” at the beginning.		40 points
Subtotal		/240 Points
Total Points		/600 Points